Find Three Largest Numbers

AlgoExpert · Easy · Searching

The problem

- Return the three largest values (sorted ascending) without sorting the input.

Intuition

- Keep a tiny sorted window of the three largest seen so far, and shift values into place as larger ones arrive.

Brute force vs optimal

- Sort the array and take the last three → O(n log n)
- Optimal: one pass with three slots → O(n)
- No sorting, no extra memory

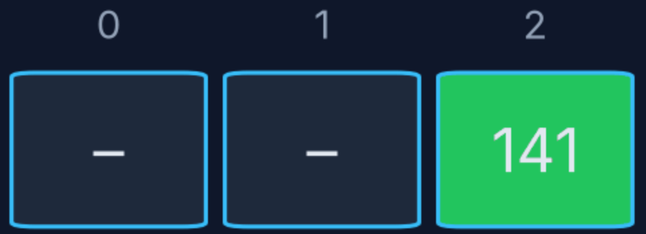
Approach

- 1. Start with three empty slots
- 2. Compare each number with the largest slot first
- 3. Shift smaller values down to make room
- 4. Return the three values ascending

Worked example



three largest (ascending)

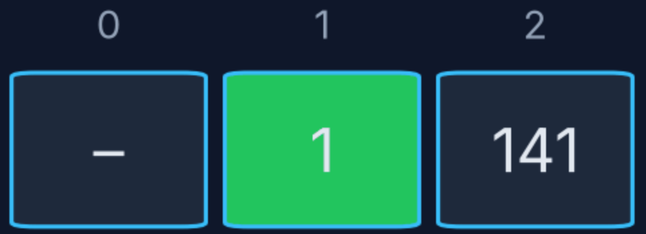


141 goes into slot 2

Worked example



three largest (ascending)

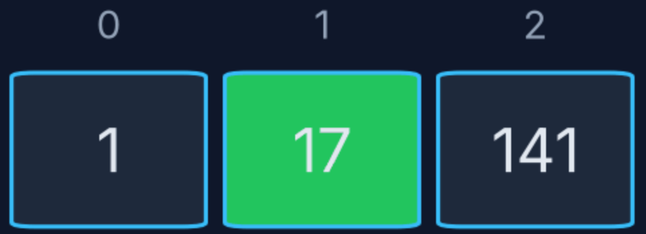


1 goes into slot 1

Worked example



three largest (ascending)

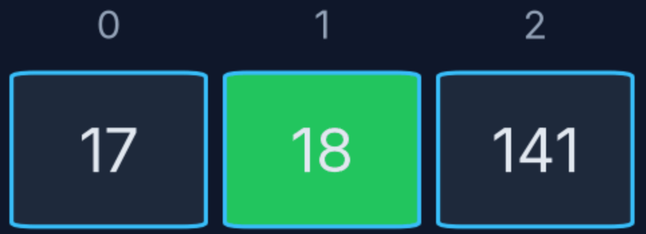


17 goes into slot 1

Worked example



three largest (ascending)

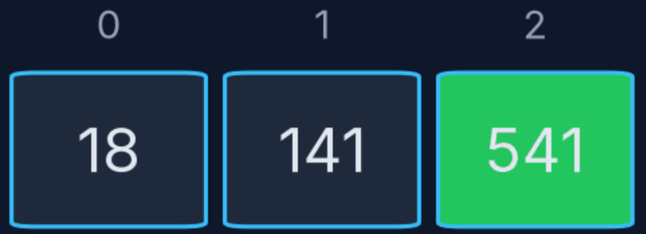


18 goes into slot 1

Worked example



three largest (ascending)



541 goes into slot 2

Java solution

```java
public class FindThreeLargestNumbers {
    public static int[] findThreeLargestNumbers(int[] array) {
        int[] top = {Integer.MIN_VALUE, Integer.MIN_VALUE, Integer.MIN_VALUE};
        for (int x : array) {
            if (x > top[2]) { top[0] = top[1]; top[1] = top[2]; top[2] = x; }
            else if (x > top[1]) { top[0] = top[1]; top[1] = x; }
            else if (x > top[0]) top[0] = x;
        }
        return top;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Duplicates can fill several slots (7, 7)
- Negative numbers: start slots as empty, not 0
- Shift order matters: move down before overwriting

Interview tips & follow-ups

- Follow-up: k largest → min-heap of size k, O(n log k)
- Quickselect for O(n) average

Recap

- Keep a tiny sorted window of the three largest seen so far, and shift values into place as larger ones arrive.
- Complexity: O(n)